



Process & Thread Creation

Chapter 4



Process Creation

- 4 Using `fork()` system call
- 4 The new process is almost identical to the original, executing the same code but with its own data space, environment, and file descriptors.

```
#include <sys/types.h>
```

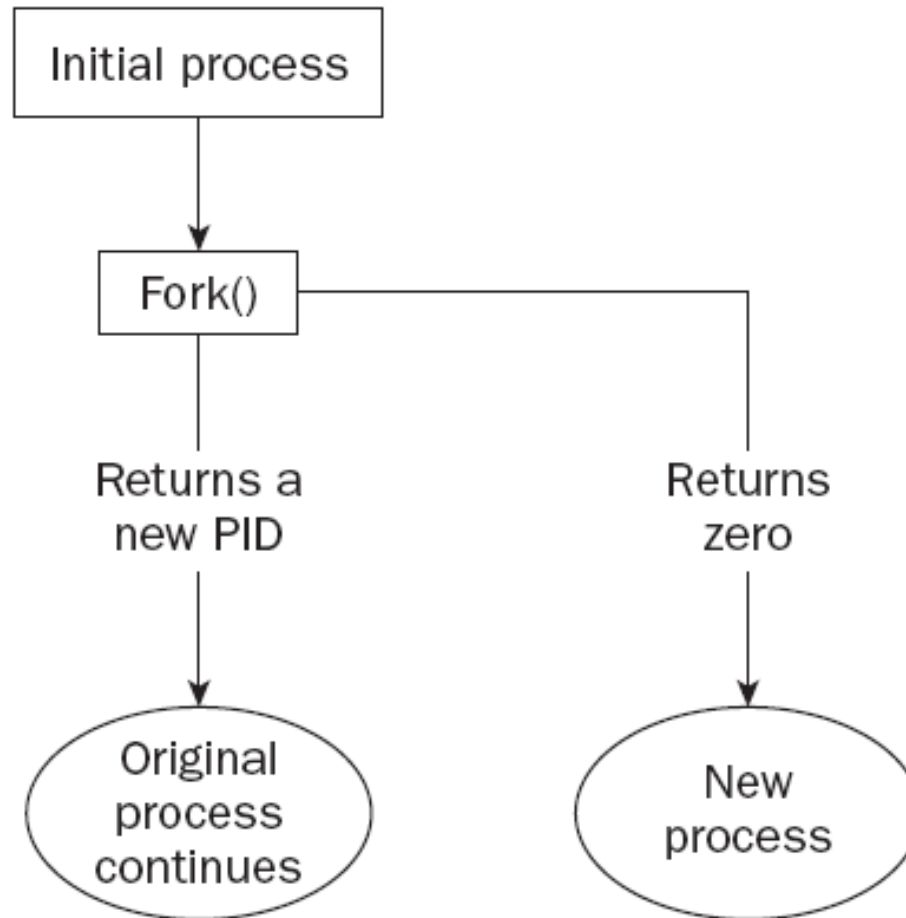
```
#include <unistd.h>
```

```
pid_t fork(void);
```

- 4 Return PID to parent and zero to new process



Process Creation





Process Creation - Example

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main()
{
    pid_t pid;
    char *message;
    int n;
    printf("fork program starting\n");
    pid = fork();
    switch(pid)
    {
        case -1:
            perror("fork failed");
            exit(1);
```



Process Creation - Example

```
case 0:
    message = "This is the child";
    n = 5;
    break;
default:
    message = "This is the parent";
    n = 3;
    break;
}
for(; n > 0; n--) {
    puts(message);
    sleep(1);
}
exit(0);
}
```



Process Creation

- 4 Parent process can wait for one of its child processes to terminate by using `wait()` system call

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
pid_t wait(int *stat_val);
```

- 4 Returns the PID of child process that terminated.
- 4 Status information (value returned) will be stored at location pointed by `stat_val`.



Process Creation

4 Macros to interpret value returned

WIFEXITED (stat_val)

- Return nonzero if the child is terminated normally.

WEXITSTATUS (stat_val)

- Returns child exit code if **WIFEXITED** is nonzero.



Process Creation - Example

```
if(pid != 0)
{
    int stat_val;
    pid_t child_pid;
    child_pid = wait(&stat_val);
    printf("Finished: PID = %d\n",
           child_pid);
    if(WIFEXITED(stat_val))
        printf("Exit code %d\n",
               WEXITSTATUS(stat_val));
    else
        printf("Terminated abnormally\n");
}
```




Thread Creation

4 Using `pthread_create()`

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread,  
    pthread_attr_t *attr,  
    void *(*start_routine) (void *),  
    void *arg);
```



Thread Creation

- 4 First argument is a pointer to `pthread_t`, use to store thread identifier.
- 4 Second argument sets the thread attributes, usually NULL.
- 4 Third argument: the function that the thread is to start executing.
- 4 The last argument: arguments that are to be passed to the function.
- 4 `pthread_create` returns 0 if success or an error number if anything goes wrong.



Thread Creation

- 4 Thread calls `pthread_exit()` when it terminates.

```
void pthread_exit(void *retval);
```

- 4 `pthread_join()` waits for a thread to finish.

```
int pthread_join(pthread_t th,  
                  void **thread_return);
```



Thread Creation - Example

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <pthread.h>

void *thread_function(void *arg);
char message[] = "Hello World";

int main()
{
    int res;
    pthread_t a_thread;
    void *thread_result;
    res = pthread_create(&a_thread, NULL,
        thread_function, (void *)message);
```



Thread Creation - Example

```
if(res != 0){
    perror("Thread creation failed");
    exit(EXIT_FAILURE);
}
printf("Waiting thread...\n");
res = pthread_join(a_thread,
                   &thread_result);

if(res != 0){
    perror("Thread join failed");
    exit(EXIT_FAILURE);
}
printf("Thread joined, returned %s\n",
       (char *)thread_result);
printf("Message is %s\n", message);
exit(EXIT_SUCCESS);
}
```



Thread Creation - Example

```
void *thread_function(void *arg)
{
    printf("thread_function is running.
           Argument: %s\n", (char *)arg);
    sleep(3);
    strcpy(message, "Bye!");
    pthread_exit("Thank you for the CPU time");
}
```



Concurrency: Mutual Exclusion and Synchronization

Chapter 5



Concurrency

4 Fundamental to operating system design

4 Arise in three contexts:

- Multiple applications
 - Multiprogramming (shared processing time)
- Structured application
 - Application can be a set of concurrent processes
- Operating-system structure
 - The same structuring advantages is applied to systems programs; Operating system is a set of processes or threads



Concurrency

4 Concurrency encompasses OS design issues including:

- Communication among processes
- Sharing of and competing for resources
- Synchronization of multiple processes
- Allocation of processor time



Terms Related to Concurrency

Critical section	A section of code within a process that requires access to shared resources and which may not be executed while another process is in a corresponding section of code.
Deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.



Terms Related to Concurrency

Mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
Race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
Starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler, although it is able to proceed, it is never chosen.



Difficulties of Concurrency

- 4 Sharing of global resources (example, two processes use the same global variable and both reads and writes on that variable, then the order in which the various reads and writes are executed is critical.)
- 4 Operating system managing the allocation of resources optimally (resource allocated and the process suspended). For example, process A may request use of, and be granted control of, a particular I/O channel and then be suspended before using that channel. It may be undesirable for the OS to simply lock the channel and prevent its use by other processes;
- 4 Difficult to locate programming errors (the results are not deterministic and reproducible)



A Simple Example

```
void echo()  
{  
    c_in = getchar();  
    c_out = c_in;  
    putchar(c_out);  
}
```

Program provides echo procedure; input is obtained from a keyboard, one keystroke at a time. Each input character is stored in variable `c_in`. It is then transferred to variable `c_out` and sent for display.



A Simple Example

In multiprocessor system, P1 and P2 both executing on separate processor concurrently:

Process P1

```
.  
c_in = getchar() ;  
.   
c_out = c_in;  
putchar(c_out) ;  
.   
.
```

Process P2

```
.   
c_in = getchar() ;  
c_out = c_in;  
.   
putchar(c_out) ;  
.
```



A Simple Example

1. Process P1 invokes **echo** procedure and interrupted immediately after **getchar**, the most recently entered character is stored in **c_in**
2. Process P2 is activated and invokes the **echo** procedure, another character is entered and displayed
3. Process P1 resume, but the first character entered has been overwritten



A Simple Example

To overcome the problem:

- 4 Process P1 and P2 are both executing, on a separate processor, P1 invokes **echo** procedure
- 4 While P1 is still inside echo procedure, P2 invokes **echo**. Because P1 is still inside **echo** procedure, P2 is blocked from entering the procedure. Therefore, P2 is suspended awaiting the availability of the **echo** procedure.
- 4 At a later time, P1 completes execution of **echo**, exits the procedure, and continues executing. Immediately upon the exit of P1 from echo, P2 is resumed and begins executing **echo**.



Example 1 (Race Condition)

- 4 Multiple processes / threads read and write data items
→ the final results depends on the order of execution of instructions → *race condition*
- 4 P1 and P2 share global variables **a**.
- 4 At some points of execution, P1 updates **a** to the value 1, and at some point of its execution, P2 updates **a** to the value 2.
- 4 Thus, the two tasks are in a race to write variable **a**. In this example, the “loser” of the race (the process that updates last) determines the final value of **a**.



Example 2 (Race Condition)

- 4 P3 and P4 share global variables **b**, and **c** (initial values **b**=1, **c**=2)
- 4 P3 executes assignment **b = b + c** and P4 executes assignment **c = b + c**.
- 4 If P3 executes its assignment first, final values are **b = 3** and **c = 5**.
- 4 If P4 executes its assignment first, final values are **b = 4** and **c = 3**.



Operating System Concerns

- 4 Keep track of various processes (Process Control Block)
- 4 Allocate and deallocate resources for each process
 - Processor time
 - Memory
 - Files
 - I/O devices
- 4 Protect data and resources against unintended interference by other process.
- 4 Output of process must be independent of the speed of execution of other concurrent processes



Competition for Resources (1)

Control problems:

1. Mutual Exclusion

- Critical sections: a program section that uses critical resource (non-sharable resource; e.g. printer)
 - Only one program at a time is allowed in its critical section
 - Example: only one process at a time is allowed to send command to the printer



Competition for Resources (2)

2. Deadlock: two processes waiting for resource owned by each other
 - E.g. Both processes, P1 and P2 needs access to two resources, R1 and R2. Then it is possible to have the following situation:
 - OS assigns R1 to P2 and R2 to P1. Each process is waiting for one of the two resources. Neither will release the resource and performed the function requiring both resources. The two processes are deadlocked.
3. Starvation: process indefinitely be denied access to resource



Requirements for Mutual Exclusion

1. Only one process at a time is allowed in the critical section for a resource
2. A process that halts in its noncritical section must not interfere other processes
3. No deadlock or starvation: A process requiring access to noncritical section must not be delayed indefinitely
4. A process must not be delayed access to a critical section when there is no other process using it
5. No assumptions are made about relative process speeds or number of processes
6. A process remains inside its critical section for a finite time only



Requirements for Mutual Exclusion

Approaches to satisfy the requirements:

- 4 Software approach → Leave the responsibility to processes that wish to execute concurrently; they coordinate with one another to enforce mutual exclusion, no support from the programming language or the operating system
 - High processing overhead and bugs
- 4 Hardware approach → use special purpose machine instructions
 - Lower overhead but not suitable for general purpose solution
- 4 Operating system or programming language support:
 - Semaphores
 - General/Counting Semaphore
 - Binary Semaphore
 - Mutex
 - Monitors
 - Message passing



Hardware Approaches

1. Interrupt Disabling
2. Special Machine Instructions



Interrupt Disabling

- 4 A process (in uniprocessor machine) runs until it invokes an operating system service or until it is interrupted
- 4 To guarantee mutual exclusion, must prevent a process from being interrupted
- 4 Problems:
 - Processor is limited in its ability to interleave programs → efficiency degraded
 - In multiprocessor machine, disabling interrupts on one processor will not guarantee mutual exclusion; disable all interrupts is costly



Special Machine Instructions

- 4 Performed in a single instruction cycle
- 4 During execution of the instruction, access to the memory location is blocked for any other instructions referencing that location
- 4 2 most common instructions:
 - Compare and Swap instruction
 - Exchange instruction



Special Machine Instructions

4 Compare and Swap Instruction

```
int compare_and_swap (int *word, int testval,  
int newval) {  
    1 int oldval;  
    oldval = *word;  
    if (oldval == testval)  
        *word = newval;  
    return oldval;  
}
```

It checks a memory location (`*word`) against a test value (`testval`). If the memory location's current value is `testval`, it is replaced with `newval`; otherwise it is left unchanged. The old memory is always returned.



Special Machine Instructions

Continue to execute to test the value of **bolt**, enter critical section if the value is 0

bolt is set to 0 when the process leaves its critical section

parbegin:suspend the execution of the main program, initiate P1, P2, ...Pn concurrently, when they have terminated, resume the main program

```
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /*do nothing*/;
        /*critical section*/
        bolt = 0;
        /*remainder*/
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

Concurrent processes



Special Machine Instructions

- 4 Exchange Instruction: exchange the contents of a register with the contents of a memory location

```
void exchange (int &register, int &memory)
{
    int temp;
    temp = memory;
    memory = register;
    register = temp;
}
```



Special Machine Instructions

```
/* program mutual exclusion */
int const n = /* number of processes**/;
int bolt;
void P(int i)
{
    while (true) {
        int keyi = 1;
        do exchange (keyi, bolt)
        while (keyi != 0);
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

(b) Exchange instruction

This shows a mutual exclusion protocol based on the use of an exchange instruction.

- A shared variable **bolt** is initialized to 0. Each process uses a local variable **key** that is initialized to 1.
- The only process that may enter its critical section is one that finds **bolt** equal to 0.
- It excludes all other processes from the critical section by setting **bolt** to 1.
- When a process leaves its critical section, it resets bolt to 0, allowing another process to gain access to its critical section.



Machine Instructions

4 Advantages

- Applicable to any number of processes on either a single processor or multiple processors sharing main memory
- It is simple and therefore easy to verify
- It can be used to support multiple critical sections
- Still have multi-programming



Machine Instructions

4 Disadvantages

- Busy-waiting consumes processor time (a process does nothing until it is permitted to enter critical section)
- Starvation is possible when a process leaves a critical section and more than one process is waiting.
- Deadlock
 - If a low priority process has the critical region and a higher priority process needs, the higher priority process will obtain the processor to wait for the critical region



OS / Programming Language Support

4 Semaphore:

- Signalling approach:
 - Use `semWait` and `semSignal` to control the value of semaphore
- Two types:
 - General/Counting Semaphore: Value could be any integer
 - Binary Semaphore: Value is either 0 or 1

4 Mutex:

- Lock-based approach:
 - The thread/process that wants to access critical section must acquire the key (mutex) before it can access it



Semaphores

- 4 Processes cooperate by means of signals → a process stop at a place until it receives a specific signal
- 4 A special variable called a *semaphore* is used for signaling
- 4 If a process is waiting for a signal, it is suspended until that signal is sent



Semaphores

- 4 Semaphore is a variable that has an integer value
 - May be *initialized* to a nonnegative number
 - *Wait* operation decrements the semaphore value. If the value is negative, the process is blocked.
 - *Signal* operation increments semaphore value. If the value is less than or equal to zero, the process blocked by *Wait* operation is unblocked.



Definition of Semaphore Primitives

```
struct semaphore {
    int count;
    queueType queue;
};

void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}

void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```



Semaphores

4 Operation as follow:

- To begin, the semaphore has a zero or positive *initial* value. If the value is +ve, the value equals the number of processes that can issue a semWait and immediately continue to execute.
- If the value is zero due to the number of processes issued semWait == *initial* value, the next process to issue a semWait will be blocked, and the semaphore value goes -ve.
- Each signal unblocks (by semSignal) one of the waiting processes when the semaphore value is -ve.



Binary Semaphore

Value of binary semaphore is either 0 (resource is unavailable) or 1 (resource is available).

- May be initialized to 0 or 1.
- **Wait** operation checks the value of semaphore. If the value is zero, the process is blocked. Otherwise, the process continues execution and the value changed to zero.
- **Signal** operation checks if any processes blocked on this semaphore. If so, a process blocked by *Wait* operation is unblocked. Otherwise, the value of semaphore is set to 1.



Binary Semaphore Primitives (Definition)

```
struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};

void semWaitB(binary_semaphore s)
{
    if (s.value == one)
        s.value = zero;
    else {
        /* place this process in s.queue */;
        /* block this process */;
    }
}

void semSignalB(semaphore s)
{
    if (s.queue is empty())
        s.value = one;
    else {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```



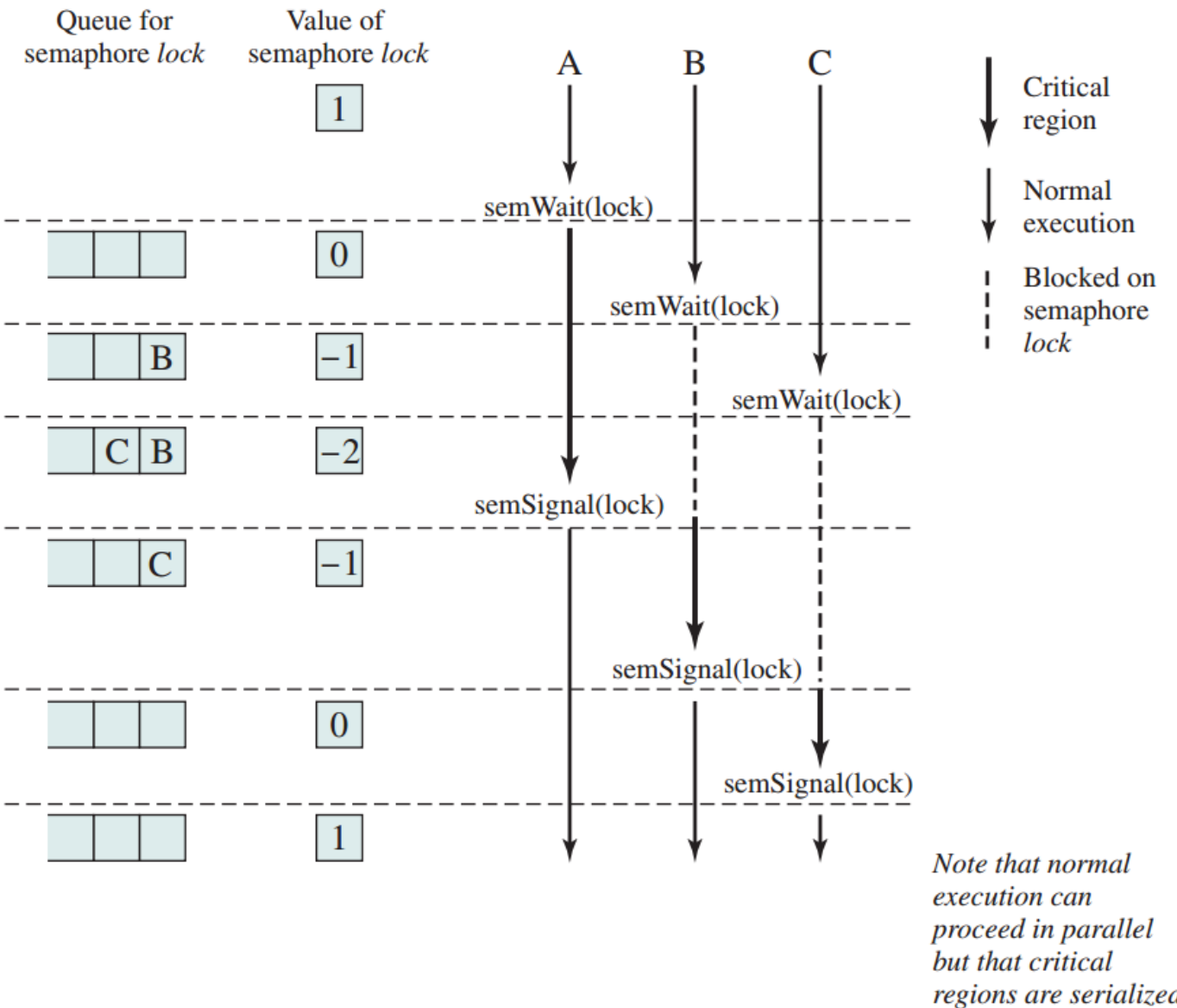
Mutual Exclusion Using Counting Semaphores

```
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
    while (true) {
        semWait(s);
        /* critical section */;
        semSignal(s);
        /* remainder */;
    }
}
void main()
{
    parbegin (P(1), P(2), . . . , P(n));
}
```

*Mutual exclusion is just a semaphore initialized to 1 as only one process can run the critical section. The main difference is, the process that issue semWait to the semaphore must also be the one that issue semSignal when it leaves the critical section.



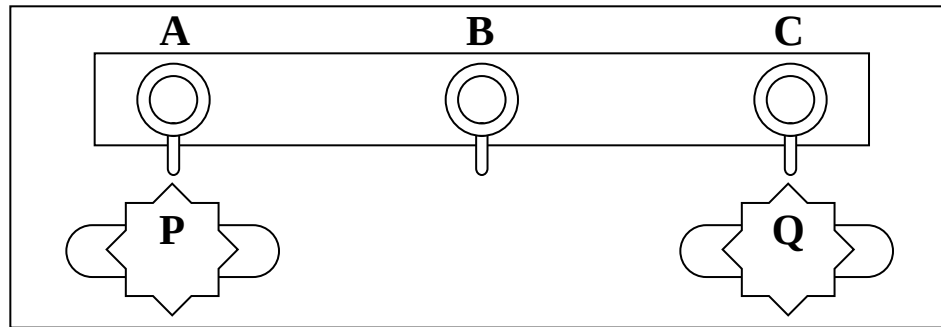
Mutual Exclusion Using Counting Semaphores





Semaphore (Example)

- 4 There are 3 frying-pans, A, B and C, and 2 chefs, P and Q.



- A new order received, chef P is required to prepare 5 sandwiches while chef Q is required to prepare 5 hotdog burgers. Pan B is shared by the chefs and only one person can use it at a time. Chef P needs pan B to fry ham and chef Q needs pan B to fry egg. The problem can be solved using binary semaphore.



Semaphore (Example)

- 4 Write the algorithms for chef P and chef Q to fry the hams and eggs respectively. Assume that
- binary semaphore ***B*** is used
 - function ***fry_ham()***: chef P fries ham using pan B
 - function ***fry_egg()***: chef Q fries egg using pan B
 - ***h*** is the number of hams fried
 - ***e*** is the number of eggs fried



Semaphore (Solution)

Chef P

```
h=0;
```

```
while(h<5)
```

```
{
```

```
    semWait(B);
```

```
    fry_ham();
```

```
    semSignal(B);
```

```
    h++;
```

```
}
```

Chef Q

```
e=0;
```

```
while(e<5)
```

```
{
```

```
    semWait(B);
```

```
    fry_egg();
```

```
    semSignal(B);
```

```
    e++;
```

```
}
```



Mutex

- Mutex is a lock that is set before a shared resource (critical section/CS) is used and is released after using it
- E.g.: thread1 run and locked the mutex (set value to 1) before entering CS.
- While thread1 yet to exit CS, the scheduler decided to context switch (e.g. time-slice of thread1 finished).
- Other thread that tries to access the same CS (now value == 1) will be put to sleep again.



Mutex

- Only thread1 can release the mutex lock. This can be done by setting the value to 0 after finish CS.
- After thread1 released the mutex lock, every other sleeping thread that want to access the same CS will be waken up. There will be a race condition on who will acquire the lock next. In other word, all of these threads will be trying to lock the mutex but only one of them will be successful to lock it.



Mutual Exclusion using Mutex

```
mutex lock;
void P(int i)
{
    mutex_lock(&lock);
    /* critical section */
    mutex_unlock(&lock);
}
void main()
{
    parbegin (P(1), P(2), ..., P(n));
}
```

POSIX says, “Mutexes have ownership, unlike semaphores. Although any thread, within the scope of a mutex, can get an unlocked mutex and lock access to the same critical section of code, only the thread that locked a mutex can unlock it.”



Producer/Consumer Problem

- General Situation:
 - One or more producers are generating data and placing these in a buffer
 - A single consumer is taking items out of the buffer one at time
 - Only one producer or consumer may access the buffer at any one time
- The Problem:
 - Ensure that the Producer can't add data into full buffer and consumer can't remove data from empty buffer



Producer and Consumer Functions

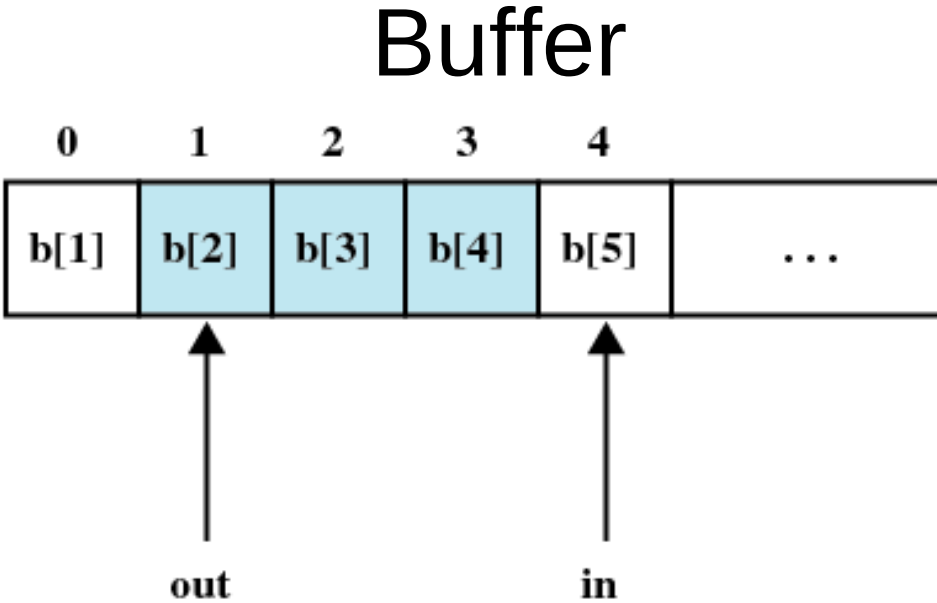
- Assume an infinite buffer ***b*** with a linear array of elements

```
producer:
while (true) {
    /* produce item v
    */
    b[in] = v;
    in++;
}
```

```
consumer:
while (true) {
    while (in <= out)
        /*do nothing */;
    w = b[out];
    out++;
    /* consume item w */
}
```



Producer/Consumer Problem



Note: shaded area indicates portion of buffer that is occupied

Figure 5.8 Infinite Buffer for the Producer/Consumer Problem



Producer/Consumer Problem

An **Incorrect** Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores

```
int n; //number of items in the buffer
binary_semaphore s = 1; //To enforce mutual ex
binary_semaphore delay = 0; //To force consumer
void producer()
{
    while (true)
    {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1)
            semSignalB(delay);
        semSignalB(s);
    }
}
```

//if $n==1$, it means before the producer produces, n was 0! It means consumer couldn't consume before this.



Producer/Consumer Problem

```
void consumer()
{
    semWaitB(delay);
    while (true)
    {
        semWaitB(s);
        take();
        n--;
        semSignalB(s);
        consume();
        if (n==0)
            semWaitB(delay);
    }
}
```

//ISSUE: when the consumer has exhausted the buffer, it needs to reset the delay semaphore so that it will be forced to wait until the producer has placed more items in the buffer.



Producer/Consumer Problem

	Producer	Consumer	s	n	Delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	if (n==1) (semSignalB(delay))		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		semSignalB(s)	1	0	0
10	semWaitB(s)		0	0	0
11	n++		0	1	0
12	if (n==1) (semSignalB(delay))		0	1	1
13	semSignalB(s)	failed	1	1	1
14		if (n==0) (semWaitB(delay))	1	1	1
15		semWaitB(s)	0	1	1
16		n--	0	0	1
17		semSignalB(s)	1	0	1
18		if (n==0) (semWaitB(delay))	1	0	0
19		semWaitB(s)	0	0	0
20		n--	0	-1	0
21		semSignalB(s)	1	-1	0



Producer/Consumer Problem

Correct Solution 1:

```
int n; //number of items in the buffer
binary_semaphore s = 1, delay = 0;
void producer()
{
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}
```

```
void consumer()
{
    int m; /* a local variable */
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        m = n;
        semSignalB(s);
        consume();
        if (m==0) semWaitB(delay);
    }
}
```



Solution by Using Counting Semaphore

```
semaphore n = 0, s = 1;
void producer()
{
    while (true) {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
```

Deadlock occurs if the **semWait(n)** and **semWait(s)** are interchanged.

Solution 2 by using Semaphore

```
void consumer()
{
    while (true) {
         semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}
```



Solution for Finite Buffer (Circular buffer)

Block on:	Unblock on:
Producer: insert in full buffer	Consumer: item inserted
Consumer: remove from empty buffer	Producer: item removed

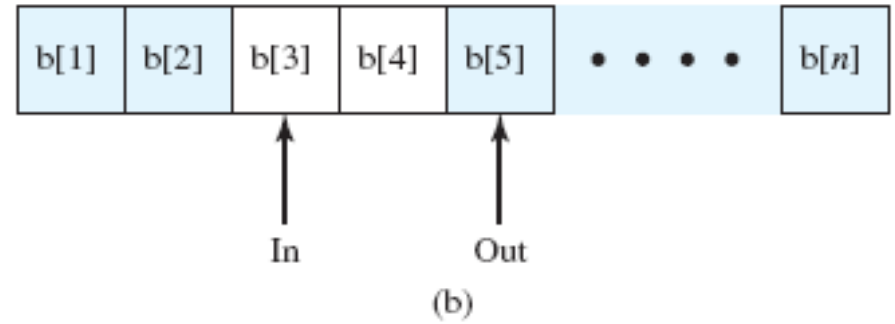
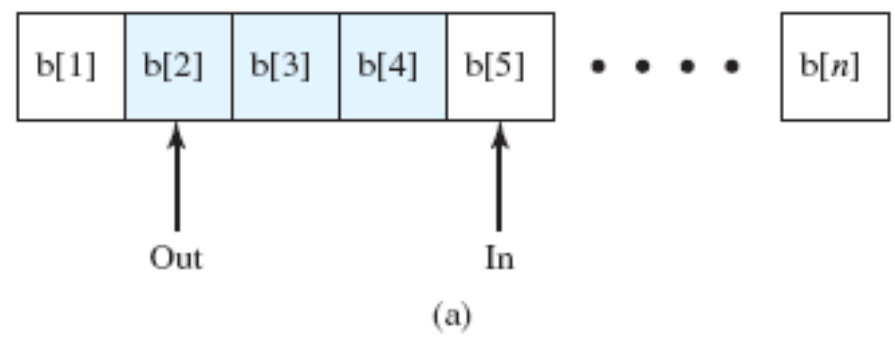


Figure 5.12 Finite Circular Buffer for the Producer/Consumer Problem



Producer with Circular Buffer

Producer

```
while (true) {  
    /* produce item v */  
    while ((in + 1) % n == out)  
        /* do nothing */;  
    b[in] = v;  
    in = (in + 1) % n  
}
```

Consumer

```
while (true) {  
    while (in == out)  
        /* do nothing */;  
    w = b[out];  
    out = (out + 1) % n;  
    /* consume item w */  
}
```



Producer/Consumer Problem: Finite Buffer

```
const int sizeofbuffer = /* buffer size */; // number of items in buffer
semaphore s = 1, n = 0, e = sizeofbuffer; // number of empty spaces
void producer()
{
    while (true) {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
```

```
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}

void main()
{
    parbegin (producer, consumer);
}
```

Solution with Bounded-Buffer.



Monitors

- 4 A high-level language constructs that provide equivalent functionality to that of semaphores but easier to control
- 4 Monitor is a software module
- 4 Chief characteristics
 - Local data variables are accessible only by the monitor
 - Process enters monitor by invoking one of its procedures
 - Only one process may be executing in the monitor at a time, others will be blocked

Monitor Structure

A process in monitor may block itself on condition x by issuing `cwait( $x$ )`.

If a process in monitor detects a change in condition variable x , it issues `csignal( $x$ )` to alert the corresponding condition queue.

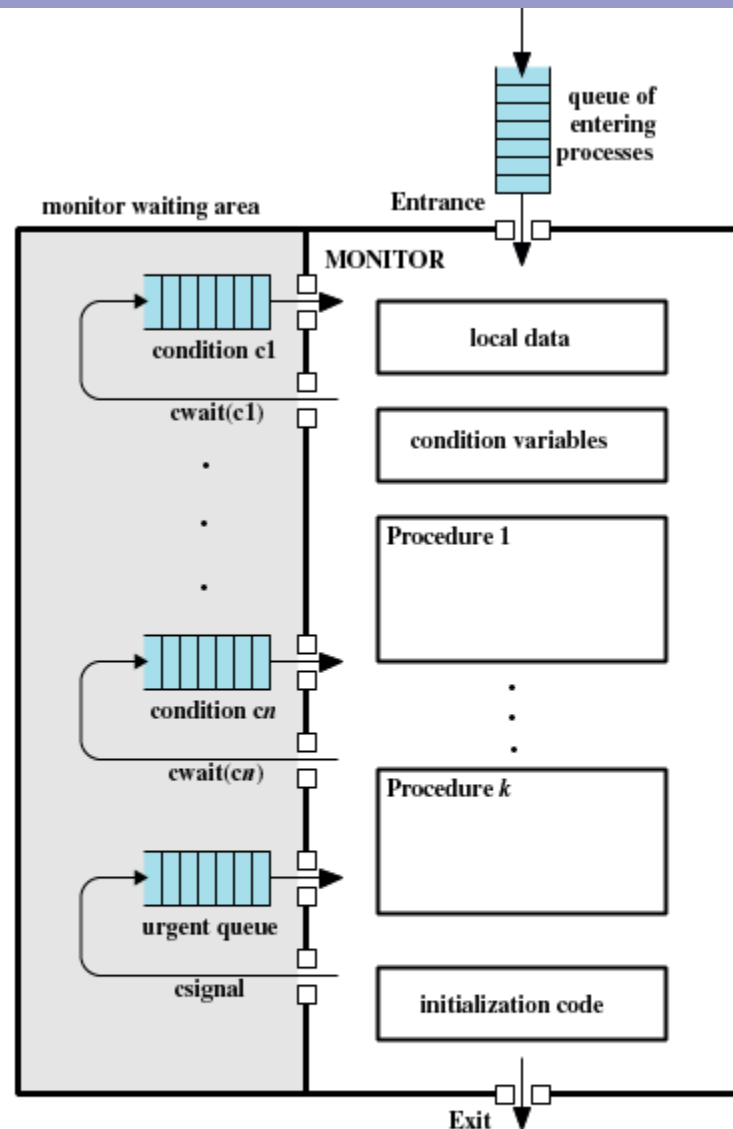


Figure 5.15 Structure of a Monitor



Monitors (Producer/Consumer)

```
monitor boundedbuffer;
char buffer [N];                                     /* space for N items */
int nextin, nextout;                                 /* buffer pointers */
int count;                                           /* number of items in buffer */
cond notfull, notempty;                             /* condition variables for synchronization */

void append (char x)
{
    if (count == N) cwait(notfull);                 /* buffer is full; avoid overflow */
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    /* one more item in buffer */
    csignal(notempty);                             /* resume any waiting consumer */
}

void take (char x)
{
    if (count == 0) cwait(notempty);                 /* buffer is empty; avoid underflow */
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    csignal(notfull);                               /* one fewer item in buffer */
    /* resume any waiting producer */
}

/* monitor body */
nextin = 0; nextout = 0; count = 0;                 /* buffer initially empty */
}
```



Monitors

```
void producer()
{
    char x;
    while (true) {
        produce(x);
        append(x);
    }
}
void consumer()
{
    char x;
    while (true) {
        take(x);
        consume(x);
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

**A Solution to the Bounded-Buffer Producer/Consumer Problem
Using a Monitor**

Producer does not have direct access to *buffer*.



Message Passing

- 4 When processes interact with one another, 2 requirements must be satisfied:
 - Synchronization → enforce mutual exclusion
 - Communication → exchange information
- 4 The actual function of message passing is normally provided in the form of a pair of primitives:
`send (destination, message)`
`receive (source, message)`
- 4 Message Passing is one solution to the second requirement
 - Added bonus: It works with shared memory *and* with distributed systems



Message Passing

Table 5.4 Design Characteristics of Message Systems for Interprocessor Communication and Synchronization

Synchronization	Format
Send	Content
blocking	Length
nonblocking	fixed
Receive	variable
blocking	
nonblocking	Queuing Discipline
test for arrival	FIFO
	Priority
Addressing	
Direct	
send	
receive	
explicit	
implicit	
Indirect	
static	
dynamic	
ownership	



Synchronization

- 4 Sender and receiver may or may not be blocking (waiting for message)
- 4 Blocking send, blocking receive: Both sender and receiver are blocked until message is delivered
- 4 Nonblocking send, blocking receive
 - Sender continues on but receiver is blocked until the requested message arrives
 - Allow a process to send message(s) to a variety of destinations as quick as possible
- 4 Nonblocking send, nonblocking receive: Neither party is required to wait



Addressing

4 Two categories:

- Direct addressing
- Indirect addressing

4 Direct addressing

- Send primitive includes a specific identifier of the destination process
- Receive primitive could know ahead of time which process a message is expected → explicitly designate a source
- Receive primitive could use source parameter to return a value when the receive operation has been performed



Addressing

4 Indirect addressing

- Messages are sent to a shared data structure consisting of queues (not to the receiver)
- Queues are called mailboxes
- One process sends a message to the mailbox and the other process picks up the message from the mailbox



Addressing

4 Relationship between senders and receivers

- One-to-one

- Allows a private communications link to be set up
- Insulates the interaction from erroneous interference from other processes

- Many-to-one

- One process provides service to a number of other processes
- Port: is a mailbox associated with one receiver and multiple senders



Addressing

4 Relationship between senders and receivers

- One-to-many
 - One sender and multiple receivers
 - Useful for applications where a message or some information is to be broadcast to a set of processes
- Many-to-many
 - Allows multiple server processes to provide concurrent service to multiple clients



Addressing

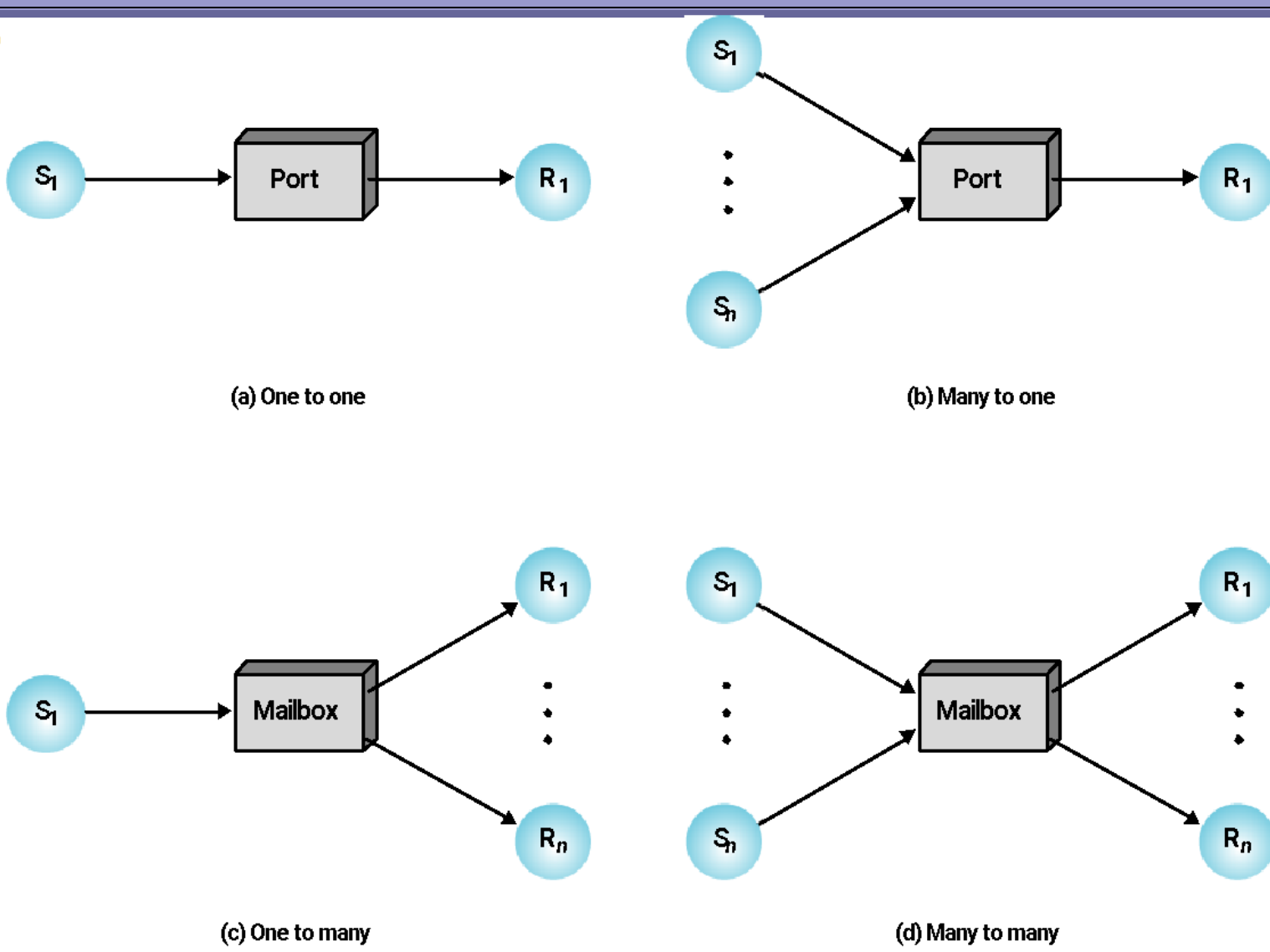


Figure 5.18 Indirect Process Communication



Message Format

- Consists of header and body of message

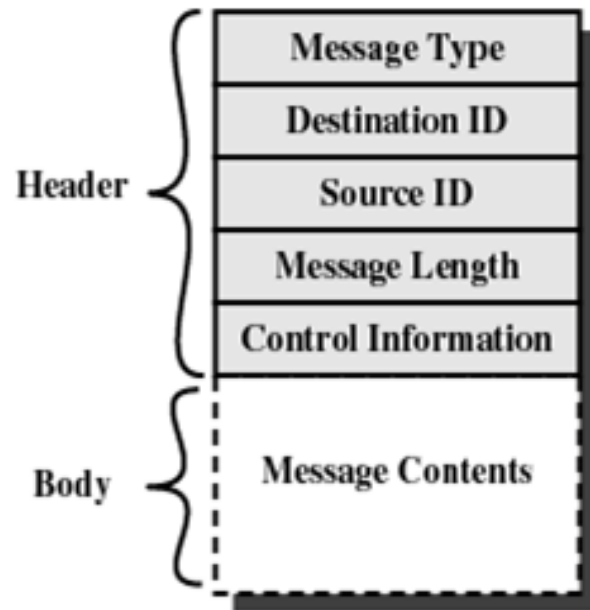


Figure 5.19 General Message Format



Mutual Exclusion Using Messages

```
const int n = /* number of processes */;
void P(int i)
{
    message msg;
    while (true) {
        receive (box, msg); //Blocking receive
        /* critical section */;
        send (box, msg); //Nonblocking send
        /* remainder */;
    }
}
void main()
{
    create_mailbox (box);
    send (box, null);
    parbegin (P(1), P(2), . . . , P(n));
}
```




Mutual Exclusion Using Messages

- If a process wants to enter its critical section, it first attempts to receive a message. If the mailbox empty, the process is blocked until it acquires a message.
- The message is placed back into mailbox after the critical section.



~ *The End* ~